HuggingFace Transformers: API, Models & Fine-tuning Techniques

Day 2 — Module 3 & Module 4

Instructor: Chandrashekar Babu <training@chandrashekar.info>

https://www.slashprog.com/ | https://www.chandrashekar.info/

ML ENGINEERING | NLP PRACTITIONERS | HUGGINGFACE

DAY 2 OVERVIEW

What We're Covering Today

Module 3

Model Architectures and Task-Specific Models

- Transformer Architecture Family Tree
- Encoder-Only Models (BERT Family)
- Decoder-Only Models (GPT Family)
- Hands-on Lab: Feature Extraction & Text Generation

Module 4

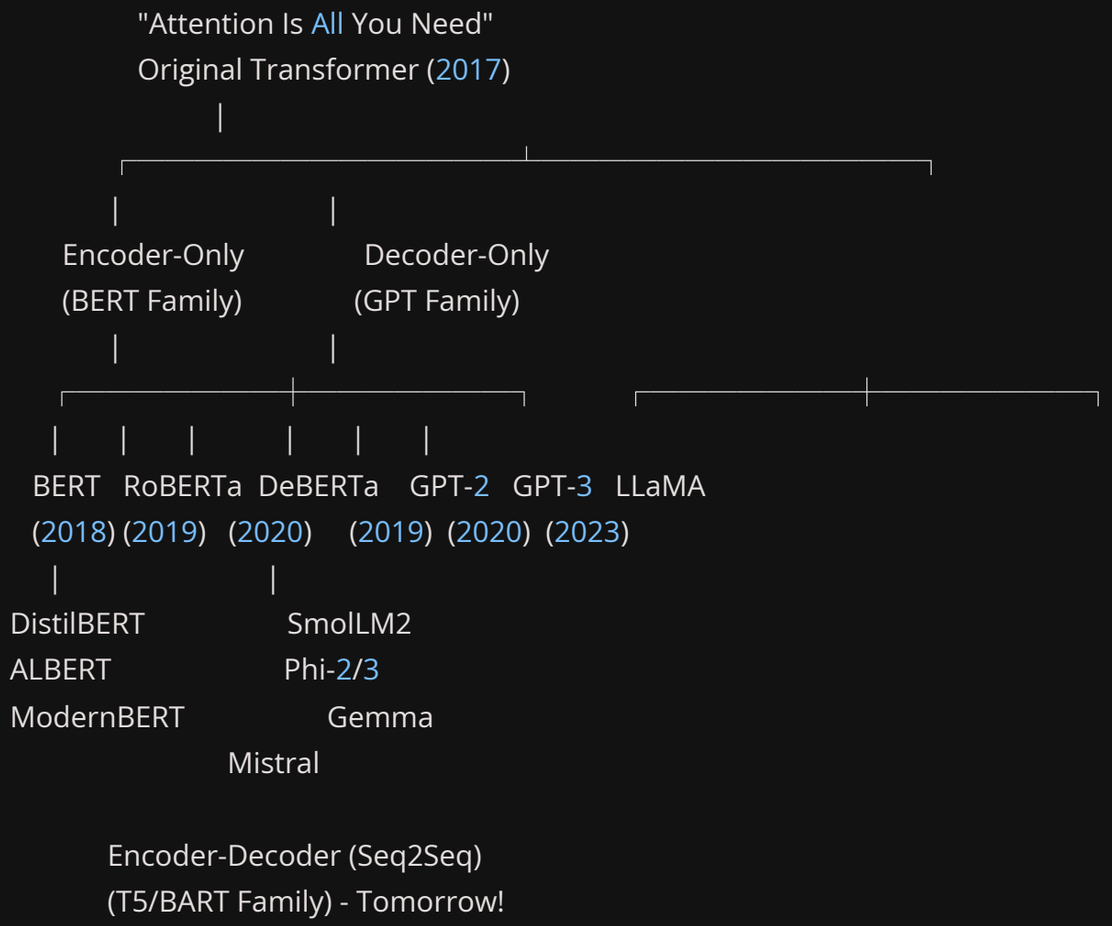
Encoder-Decoder and Multimodal Models

- T5, BART and Encoder-Decoder Architecture
- Vision Transformers (ViT)
- Multimodal Models: CLIP and Vision-Language
- Hands-on Lab: Translation, Summarisation, Image Classification

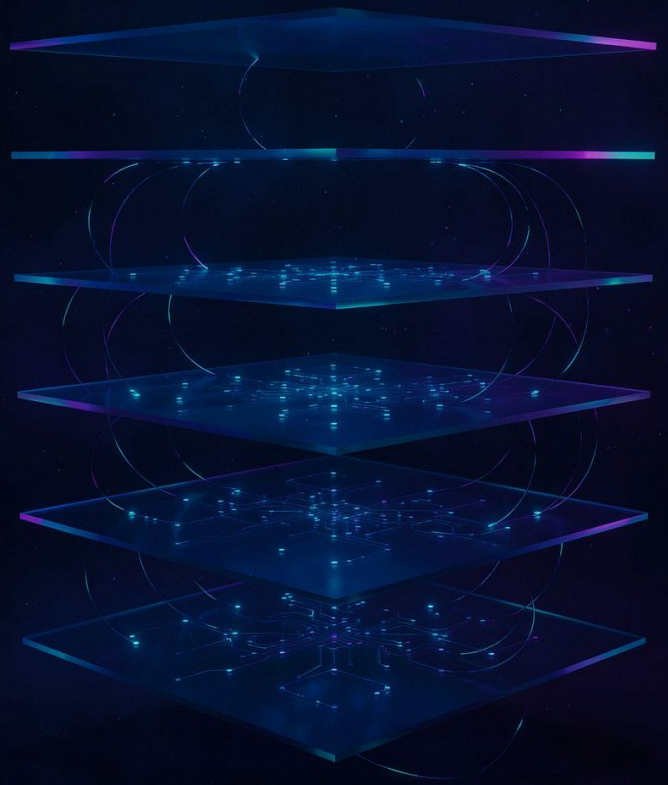
MODULE 3

The Transformer Family Tree

The 2017 paper *"Attention Is All You Need"* introduced the original Transformer. From this single architecture, two dominant families emerged — each optimised for fundamentally different tasks.



Encoder-Decoder (T5/BART) models combine the best of both worlds — covered in Module 4.

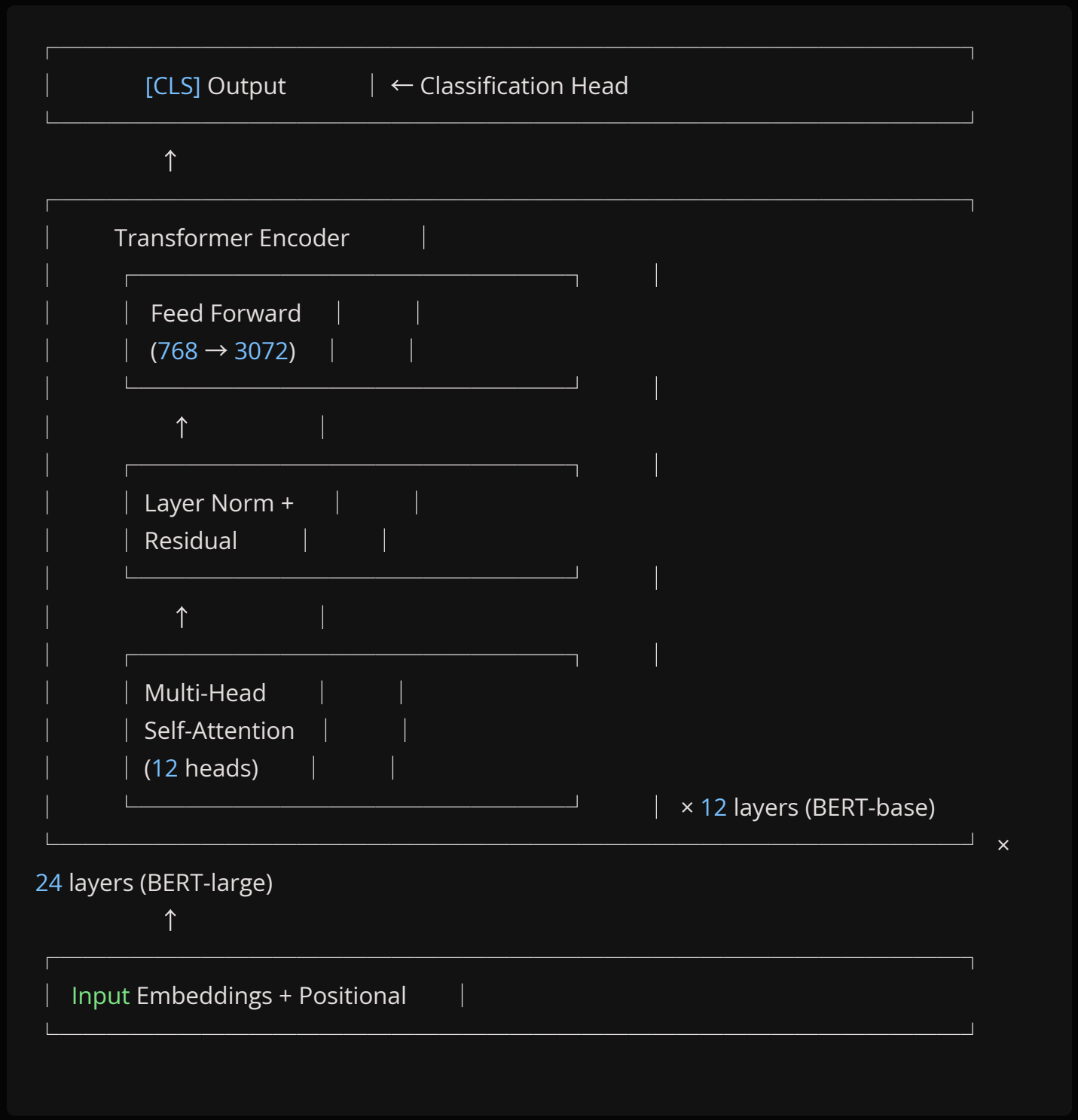


MODULE 3 — ENCODER-ONLY

Encoder-Only Architecture: The BERT Blueprint

BERT (Bidirectional Encoder Representations from Transformers) reads the **entire input sequence simultaneously** using bidirectional self-attention — meaning every token can attend to every other token. This makes BERT exceptionally powerful for **understanding** tasks.

BERT Internal Architecture



Key Architectural Points

Each Transformer Encoder block consists of:

1. **Multi-Head Self-Attention** — 12 attention heads in BERT-base, enabling the model to focus on different parts of the sequence simultaneously
2. **Layer Norm + Residual connection** — stabilises training and enables gradients to flow
3. **Feed-Forward Network** — expands from 768 → 3072 → 768 dimensions, adding representational capacity

The `[CLS]` token aggregates the entire sequence representation and is used for classification tasks.

BERT Architecture Details

BERT-base vs BERT-large

Parameter	BERT-base	BERT-large
Layers	12	24
Hidden Size	768	1024
Attention Heads	12	16
Parameters	110M	340M
Max Sequence Length	512	512
Training Data	BooksCorpus + Wikipedia (16GB)	Same

Input Representation

BERT sums three separate embedding types for each token before feeding into the encoder:

```
Input: "[CLS] The cat sat [SEP] It was tired [SEP]"

Token Emb: E[CLS] E_The E_cat E_sat E_[SEP] E_It E_was E_tired E_[SEP]
Segment Em: EA EA EA EA EA EB EB EB EB
Position Em: E0 E1 E2 E3 E4 E5 E6 E7 E8
──────────────────────────────────────────────────────────────────────────────────────────────────────
Final Input: H[CLS] H_The H_cat H_sat H_[SEP] H_It H_was H_tired H_[SEP]
```

BERT's Pre-training Objectives

1. Masked Language Modelling (MLM)

15% of input tokens are randomly selected. Of those masked tokens:

- **80%** replaced with `[MASK]`
- **10%** replaced with a random token
- **10%** kept unchanged

```
Original: "The cat sat on the mat"
Masked:   "The cat [MASK] on the [MASK]"

80%: "The cat [MASK] on the [MASK]"
10%: "The cat dog  on the mat"
10%: "The cat sat  on the mat"
```

The model predicts masked tokens using **bidirectional context** — both left and right sides of each mask.

2. Next Sentence Prediction (NSP)

BERT learns sentence-level relationships by predicting whether sentence B follows sentence A naturally.

How BERT Processes Text: A Step-by-Step Example

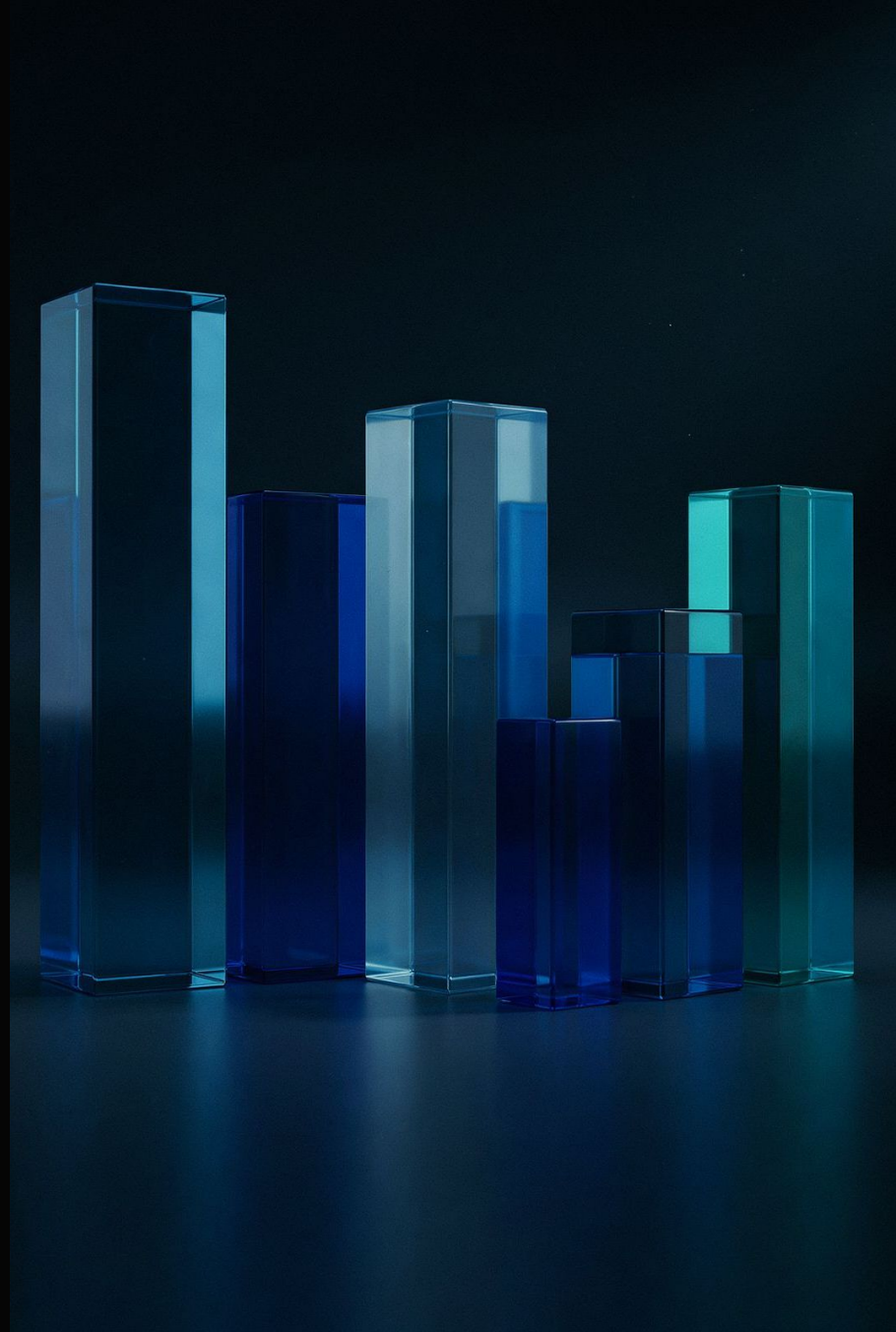
Tracing what happens inside BERT when processing `"I love machine learning!"`:

```
# Step 1: Tokenisation
tokens = ['[CLS]', 'i', 'love', 'machine', 'learning', '!', '[SEP]']
token_ids = [ 101, 1045, 2293, 3698, 4083, 999, 102 ]

# Step 2: Embedding Layer
# Each token → 768-dimensional vector
embeddings = embedding_layer(token_ids) # Shape: [7, 768]

# Step 3: 12 Transformer Layers
hidden_states = embeddings
for layer in range(12):
 # Self-attention: each token attends to ALL other tokens (bidirectional)
 attention_output = self_attention(hidden_states)
 # Feed-forward transformation
 hidden_states = feed_forward(attention_output)

# Step 4: Output
# hidden_states[0] = [CLS] token → sequence-level representation
# hidden_states[1:] = token-level representations (one per input token)
```

BERT FAMILY

BERT Model Family: Key Variants

The original BERT has spawned a rich ecosystem of improved variants, each addressing a specific limitation or use-case trade-off.



RoBERTa — Robustly Optimised BERT Approach

RoBERTa revisited BERT's training recipe and found significant headroom by training **longer on more data**.

10× More Data

160GB vs BERT's 16GB of text

Dynamic Masking

Different masks applied each epoch

No NSP

Removed Next Sentence Prediction

Byte-level BPE

Better tokenisation strategy

DistilBERT — Distilled BERT

Uses **knowledge distillation**: a smaller "student" model trained to mimic a larger "teacher" model.

```
BERT-teacher (110M params)
        ↓
DistilBERT-student (66M params)
```

- 40% smaller, 60% faster
- Retains 97% of BERT's performance
- Same architecture, 6 layers instead of 12
- Ideal for resource-constrained or latency-sensitive deployments

More BERT Variants: ALBERT, DeBERTa & ModernBERT

ALBERT

Factorised Embedding Parameterisation: Rather than a full V×H parameter matrix, ALBERT uses V×E + E×H where E ≪ H (e.g. 128 vs 768).

Cross-Layer Parameter Sharing: All 12 layers share the same weights — 18× fewer parameters than BERT-large with comparable performance.

DeBERTa

Disentangled Attention separates content and positional information using distinct weight matrices:

- Content attention (what the word means)
- Position attention (where the word appears)

Also adds an Enhanced Mask Decoder for MLM. Result: **surpasses human baseline on SuperGLUE**.

ModernBERT (2024)

BERT's modern successor, rebuilt for efficiency and scale:

- 8192-token context (vs 512)
- Flash Attention 2 & RoPE positional embeddings
- Trained on 2T tokens of modern web data
- 2× faster than DeBERTaV3
- State-of-the-art on retrieval and classification

Variants: **ModernBERT-base** (149M) and **ModernBERT-large** (395M).

BERT for Different Tasks (1 & 2)

1. Text Classification

The [CLS] token output is fed into a linear classification head. Fine-tune the full model end-to-end.

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2  # Binary classification
)

# Architecture:
# [CLS] token → Linear(num_labels) → Softmax
```

2. Named Entity Recognition (NER)

Each token gets its own prediction — using BIO tagging scheme for entity spans.

```
from transformers import AutoModelForTokenClassification

model = AutoModelForTokenClassification.from_pretrained(
 "bert-base-uncased",
 num_labels=9 # BIO tags: B-PER, I-PER, B-ORG...
)

# Input: [CLS] I love New York [SEP]
# Output: O O O B-LOC I-LOC
```

BERT for Different Tasks (3 & 4)

3. Question Answering

BERT receives both the question and context as a single input. Each token produces a start and end logit, and the highest-scoring span is extracted as the answer.

```
from transformers import AutoModelForQuestionAnswering

model = AutoModelForQuestionAnswering.from_pretrained(
    "bert-base-uncased"
)
# Input:  [CLS] Question [SEP] Context [SEP]
# Question: "What is the capital of France?"
# Context:  "France is in Europe. Its capital is Paris."
# Output: Start position + End position → "Paris"
```

4. Sentence Embeddings (via Pooling)

Two common strategies for collapsing the token-level outputs into a single fixed-size vector:

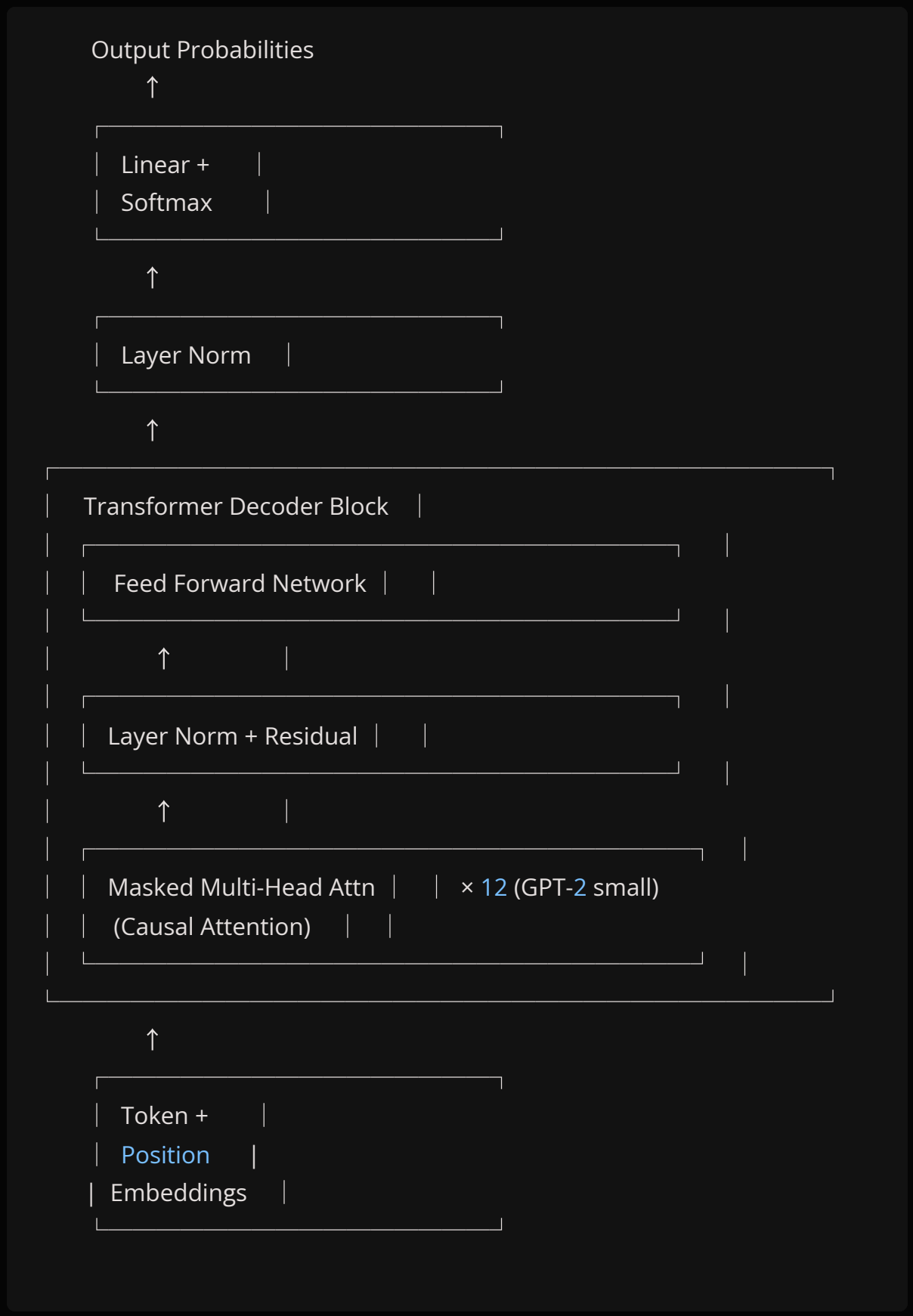
```
outputs = model(**inputs)

# Option 1: CLS token pooling
sentence_embedding = outputs.last_hidden_state[:, 0, :]

# Option 2: Mean pooling (often better)
attention_mask = inputs['attention_mask']
embeddings = outputs.last_hidden_state
mask_expanded = attention_mask.unsqueeze(-1) \
 .expand(embeddings.size()).float()
sentence_embedding = (
 (embeddings * mask_expanded).sum(1)
 / mask_expanded.sum(1)
)
```

MODULE 3 — DECODER-ONLY

Decoder-Only Architecture: The GPT Blueprint

GPT uses **causal (masked) self-attention** — each token can only attend to previous tokens. This constraint makes the architecture ideal for **text generation**: predict the next token, append it, and repeat.

GPT Internal Architecture



Key Difference vs BERT

The **causal mask** prevents any token from attending to future positions. This enforces the autoregressive property:

BERT

Bidirectional — every token sees every other token. Good for understanding.

GPT

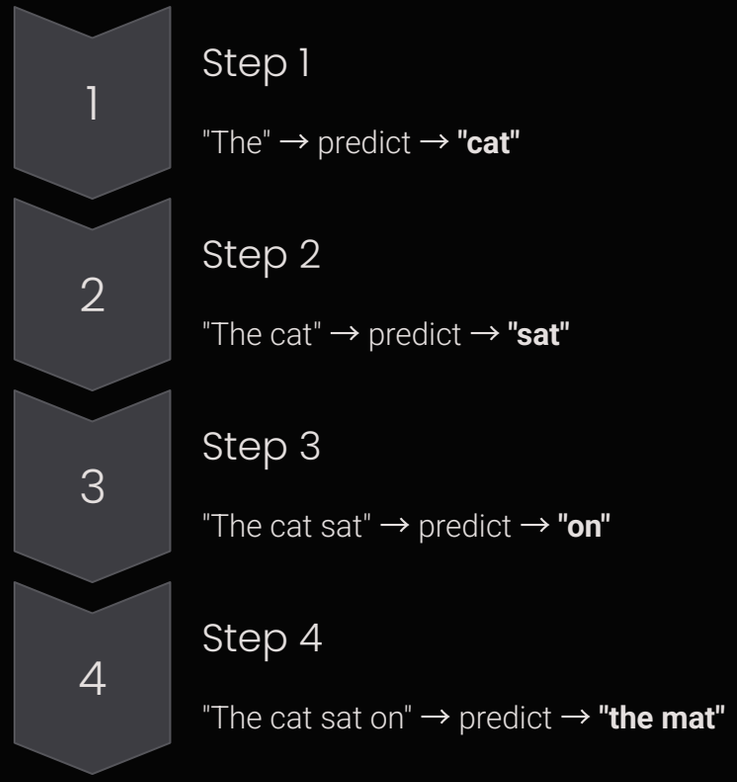
Causal (left-to-right only). Each token only sees itself and previous tokens. Good for generation.

Autoregressive Generation

How GPT Generates Text — Step by Step

```
prompt = "The cat"

# Step 1: Encode prompt
input_ids = tokenizer.encode("The cat")
# → [464, 3797]

# Step 2: Forward pass — predict next token
output = model(input_ids)
logits = output.logits[:, -1, :]  # Last token's logits
next_token_id = torch.argmax(logits)  # e.g. 318 → "sat"

# Step 3: Append and repeat
input_ids = [464, 3797, 318]  # "The cat sat"
output = model(input_ids)
next_token_id = torch.argmax(
    output.logits[:, -1, :]
)  # e.g. 319 → "on"

# Continue until max_length or EOS token
```

Visual Generation Trace



Causal Attention Mask

Understanding the Triangular Mask

The mask enforces the rule: token at position *i* can only attend to positions 0 through *i*. Positions after *i* are set to `−∞`, which becomes 0 after softmax — effectively ignoring future tokens.

Query: What can "sat" attend to?

Without mask: sat → The: ✓ cat: ✓ sat: ✓ on: ✓ the: ✓ mat: ✓
With mask: sat → The: ✓ cat: ✓ sat: ✓ on: the: mat:

Attention Matrix (0 = attend, −∞ = masked):

	The	cat	sat	on	the	mat
The	0	-∞	-∞	-∞	-∞	-∞
cat	0	0	-∞	-∞	-∞	-∞
sat	0	0	0	-∞	-∞	-∞
on	0	0	0	0	-∞	-∞
the	0	0	0	0	0	-∞
mat	0	0	0	0	0	0

✓ = 0 (attend) = −∞ (masked, becomes 0 after softmax)

GENERATION STRATEGIES

Generation Strategies

1. Greedy Search

```
output = model.generate(
    input_ids,
    max_length=50,
    do_sample=False,
    num_beams=1
)
```

✅ Fast, deterministic
❌ Repetitive, lacks creativity

2. Beam Search

```
output = model.generate(
    input_ids,
    max_length=50,
    num_beams=5,
    early_stopping=True
)
```

Keeps top-5 probable sequences at each step.
✅ Better quality than greedy
❌ Expensive, can still repeat

3. Sampling

```
output = model.generate(
    input_ids,
    max_length=50,
    do_sample=True,
    temperature=0.7
)
```

Randomly samples from the probability distribution.
✅ Creative, diverse outputs
❌ Can be incoherent at high temperature

Generation Parameters Deep Dive

Temperature

Controls the sharpness of the output distribution. Low = deterministic. High = creative/chaotic.

```
# Low temperature (deterministic)
temperature=0.1
# "The cat sat on the mat. The cat sat on the mat..."

# Medium temperature (balanced)
temperature=0.7
# "The cat sat on the mat, looking around curiously."

# High temperature (creative/chaotic)
temperature=1.5
# "The cat sat on philosophical quandaries about existence."
```

Top-K Sampling

```
top_k=50
# Only sample from the top 50 most probable tokens.
# Prevents choosing very unlikely tokens.
```

Top-P (Nucleus) Sampling

```
top_p=0.9
# Sample from the smallest set of tokens whose
# cumulative probability exceeds 0.9.
# Dynamically adjusts vocabulary size based
# on model confidence.
```

Top-P (nucleus sampling) is recommended by many practitioners as a better alternative to top-K because it adapts to the model's confidence at each step.

Combining Generation Parameters

Best Practices Recipe

Creative Writing

```
output = model.generate(
    input_ids,
    max_length=100,
    do_sample=True,
    temperature=0.8,
    top_p=0.92,
    top_k=50,
    repetition_penalty=1.2,  # Penalise repeated tokens
    no_repeat_ngram_size=3,  # No 3-gram repeats
)
```

Factual Responses

```
output = model.generate(
    input_ids,
    max_length=100,
    do_sample=True,
    temperature=0.3,   # Lower = more factual
    top_p=0.9,
    repetition_penalty=1.1,
)
```

Deterministic Tasks

```
output = model.generate(
    input_ids,
    max_length=100,
    num_beams=5,
    early_stopping=True,
    no_repeat_ngram_size=2,
)
```

Rule of Thumb

Lower temperature + beam search for structured outputs. Higher temperature + top-p for open-ended generation.

GPT Model Variants for Local Execution

All of the following models can run on consumer-grade hardware. Choose based on your VRAM budget and quality requirements.

Model	Parameters	Min VRAM	Speed	Quality
GPT-2	124M	2GB	⚡⚡⚡⚡⚡	★★
GPT-2 Medium	355M	4GB	⚡⚡⚡⚡	★★★
SmolLM2-135M	135M	1GB	⚡⚡⚡⚡⚡	★★★
SmolLM2-360M	360M	2GB	⚡⚡⚡⚡	★★★★
SmolLM2-1.7B	1.7B	4GB	⚡⚡⚡	★★★★
Phi-2	2.7B	6GB	⚡⚡	★★★★★
Phi-3-mini	3.8B	8GB	⚡⚡	★★★★★
Gemma-2B	2B	4GB	⚡⚡⚡	★★★★

Model Selection Guide

Use this decision tree to pick the right model family for your task and hardware constraints:

```
Q: What's your task?
│
├── Understanding text (classification, NER, embeddings)
│   └── Use: Encoder-only (BERT family)
│       ├── 512 tokens enough? → BERT/DistilBERT
│       ├── Need longer context? → ModernBERT (8192 tokens)
│       └── Limited resources? → DistilBERT (66M)
│
├── Generating text (completion, chat, creative)
│   └── Use: Decoder-only (GPT family)
│       ├── < 4GB VRAM → GPT-2, SmolLM2-360M
│       ├── 4-8GB VRAM → SmolLM2-1.7B, Gemma-2B
│       └── 8-16GB VRAM → Phi-3-mini, Llama-3.2-3B
│
└── Sequence-to-sequence (translation, summarization)
    └── Use: Encoder-Decoder (T5/BART) ← (Module 4)
```

HANDS-ON LAB MODULE 3

Lab: Feature Extraction with BERT

Exercise 1 — Getting Contextual Embeddings

This exercise demonstrates that the **same word receives different embeddings** depending on context — the core power of contextual representations.

```
import torch
from transformers import AutoTokenizer, AutoModel

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

sentences = [
 "I love programming in Python.",
 "The snake python is a non-venomous constrictor.",
 "Python is both a programming language and a snake species."
]

inputs = tokenizer(sentences, padding=True, truncation=True,
 return_tensors="pt")

with torch.no_grad():
 outputs = model(**inputs)

last_hidden_states = outputs.last_hidden_state # [3, seq_len, 768]
cls_embeddings = last_hidden_states[:, 0, :] # [3, 768]

# "python" embedding differs across contexts!
for i, sent in enumerate(sentences):
 tokens = tokenizer.tokenize(sent)
 if 'python' in tokens:
 idx = tokens.index('python') + 1 # +1 for [CLS]
 print(f"Sentence: {sent}")
 print(f" python embedding shape: {last_hidden_states[i, idx].shape}")
 print(f" First 5 values: {last_hidden_states[i, idx, :5].tolist()}\n")
```

Lab: Cosine Similarity of Embeddings

Exercise 2 — Comparing "python" Across Contexts

```
import torch.nn.functional as F

def get_word_embedding(model, tokenizer, sentence, word):
 inputs = tokenizer(sentence, return_tensors="pt")
 with torch.no_grad():
 outputs = model(**inputs)
 tokens = tokenizer.tokenize(sentence)
 word_tokens = tokenizer.tokenize(word)
 for i in range(len(tokens) - len(word_tokens) + 1):
 if tokens[i:i + len(word_tokens)] == word_tokens:
 # Average embeddings for subword tokens
 return outputs.last_hidden_state[
 0, i + 1:i + 1 + len(word_tokens)
 ].mean(0)
 return None

emb1 = get_word_embedding(model, tokenizer,
 "I love programming in Python.", "python")
emb2 = get_word_embedding(model, tokenizer,
 "The python snake is dangerous.", "python")
emb3 = get_word_embedding(model, tokenizer,
 "I code in Python.", "python")

sim_12 = F.cosine_similarity(emb1.unsqueeze(0), emb2.unsqueeze(0))
sim_13 = F.cosine_similarity(emb1.unsqueeze(0), emb3.unsqueeze(0))
sim_23 = F.cosine_similarity(emb2.unsqueeze(0), emb3.unsqueeze(0))

print(f"Similarity — Programming Python vs Snake Python: {sim_12.item():.3f}")
print(f"Similarity — Programming Python vs Code Python: {sim_13.item():.3f}")
print(f"Similarity — Snake Python vs Code Python: {sim_23.item():.3f}")
print("\nNote: Context matters! Same word, very different embeddings.")
```

Lab: Text Generation with GPT-2

Exercise 3 — Comparing Generation Strategies

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model_name = "gpt2" # 124M parameters — runs on CPU!
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token # GPT-2 has no pad token by default

prompt = "In a distant future, artificial intelligence"
strategies = {
 "Greedy": {
 "do_sample": False, "max_length": 60, "num_beams": 1
 },
 "Beam Search (k=5)": {
 "do_sample": False, "max_length": 60,
 "num_beams": 5, "early_stopping": True
 },
 "Sampling (temp=0.7)": {
 "do_sample": True, "max_length": 60,
 "temperature": 0.7, "top_k": 0
 },
 "Sampling (temp=1.0, top_p=0.9)": {
 "do_sample": True, "max_length": 60,
 "temperature": 1.0, "top_p": 0.9, "top_k": 50
 }
}

for strategy, params in strategies.items():
 inputs = tokenizer(prompt, return_tensors="pt")
 outputs = model.generate(**inputs, **params)
 generated = tokenizer.decode(outputs[0], skip_special_tokens=True)
 print(f"\n{'='*50}\n{strategy}:\n{'='*50}\n{generated}\n")
```

Lab: Comparing Generation Parameters

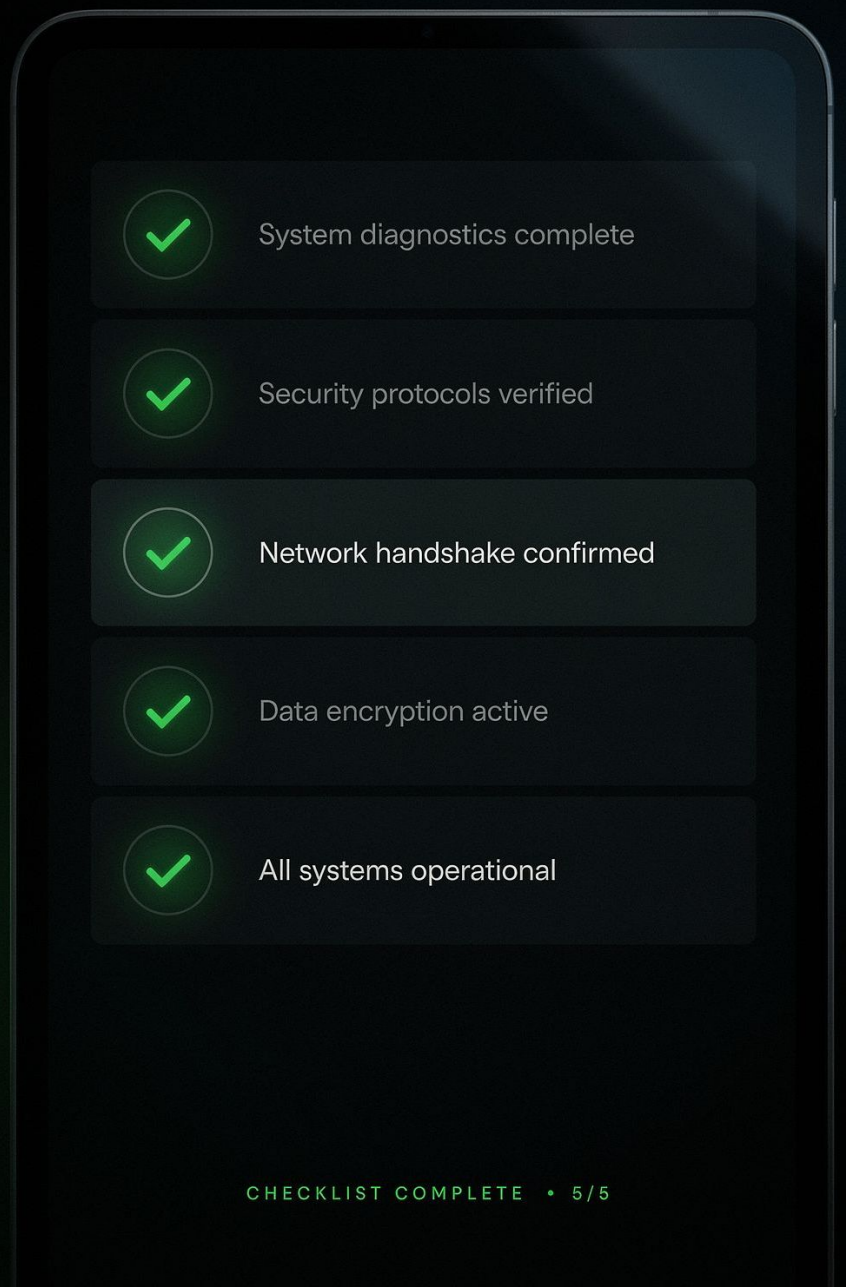
Exercise 4 — Effect of Temperature

```
prompt = "Once upon a time"
temperatures = [0.1, 0.3, 0.5, 0.7, 0.9, 1.2, 1.5]

print("Effect of Temperature on Generation:\n")
for temp in temperatures:
 inputs = tokenizer(prompt, return_tensors="pt")
 outputs = model.generate(
                **inputs,
                max_length=50,
                do_sample=True,
                temperature=temp,
                top_k=50,
                repetition_penalty=1.1,
                pad_token_id=tokenizer.eos_token_id
 )
 generated = tokenizer.decode(outputs[0], skip_special_tokens=True)
 print(f"Temperature {temp}: {generated}\n")

# Expected observations:
# - Low temp (0.1–0.3): Repetitive, deterministic, safe word choices
# - Medium temp (0.5–0.9): Coherent and natural sounding
# - High temp (1.2–1.5): Creative but potentially incoherent
```

Lab: Comparing Model Sizes

Exercise 5 — GPT-2 vs DistilGPT-2

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import time

models_to_test = [
 ("gpt2", "GPT-2 (124M)"),
 ("distilgpt2", "DistilGPT-2 (82M)"),
]

for model_name, display_name in models_to_test:
 try:
    print(f"\nLoading {display_name}...")
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name)

    prompt = "The future of AI is"
    inputs = tokenizer(prompt, return_tensors="pt")

    start = time.time()
    outputs = model.generate(
        **inputs, max_length=50,
        do_sample=True, temperature=0.7, top_p=0.9
    )
    elapsed = time.time() - start

    params = sum(p.numel() for p in model.parameters())
    generated = tokenizer.decode(outputs[0], skip_special_tokens=True)

    print(f" Parameters: {params:,}")
    print(f" Generation time: {elapsed:.2f}s")
    print(f" Generated: {generated[:100]}...")

    # Clean up memory
    del model, tokenizer
    torch.cuda.empty_cache() if torch.cuda.is_available() else None

except Exception as e:
    print(f" Failed: {e}")
```



MODULE 3 SUMMARY

Key Takeaways — Module 3

Encoder-only models (BERT)

Use **bidirectional attention** for understanding tasks. MLM pre-training enables deep contextual language understanding. Variants — RoBERTa, DistilBERT, ModernBERT — each optimise different dimensions.

Decoder-only models (GPT)

Use **causal/masked attention** for generation. Autoregressive: predict one token at a time. Generation strategies — greedy, beam search, sampling with temperature/top-k/top-p — control creativity vs quality.

Model selection

Understanding → Encoder models. Generation → Decoder models. Local execution → Small models (GPT-2, SmolLM2, Phi, Gemma).

Contextual embeddings

The **same word can have very different embeddings** depending on surrounding context. This is the core innovation of deep bidirectional transformers.

MODULE 4

Module 4: Encoder-Decoder and Multimodal Models

T5, BART, Vision Transformers, and Beyond

01

Encoder-Decoder Architecture Overview

Combining bidirectional understanding with autoregressive generation

02

T5 & BART

Text-to-text framing, span corruption, and denoising pre-training

03

Use Cases

Translation, summarisation, and sequence-to-sequence tasks

04

Vision Transformers (ViT)

Applying self-attention to image patches

05

Multimodal Models: CLIP & Vision-Language

Zero-shot classification and cross-modal retrieval

06

Hands-on Lab

Translation, summarisation, image classification, and pipeline chaining

Encoder-Decoder Architecture: The Complete Picture

Why Encoder-Decoder? The Best of Both Worlds

Aspect	Encoder-Only (BERT)	Decoder-Only (GPT)	Encoder-Decoder (T5/BART)
Input Processing	Bidirectional	Causal only	Bidirectional (via encoder)
Output Generation	Classification only	Autoregressive	Autoregressive (via decoder)
Input-Output Length	Same length	Single stream	Different lengths possible

Perfect for Tasks Where Input ≠ Output Length

Translation

Different languages naturally have different lengths and word orderings.

Summarisation

Long document input → concise summary output.

Question Answering

Short question + long context → extracted answer span.

T5

T5: Text-to-Text Transfer Transformer

T5's core philosophy is elegantly simple: **every NLP task can be framed as converting one text string into another**. This unification allows a single model and a single training objective to handle classification, generation, translation, and more.

T5 — Core Philosophy: All Tasks Are Text-to-Text

Translation:
Input: "translate English to French: Hello, how are you?"
Output: "Bonjour, comment allez-vous?"

Summarisation:
Input: "summarize: The transformer model, introduced in 2017..."
Output: "Transformer uses self-attention for parallel processing."

Classification:
Input: "sentiment: I love this product!"
Output: "positive"

Question Answering:
Input: "question: What is the capital? context: France's capital is Paris."
Output: "Paris"

Grammar Correction:
Input: "grammar: He have been working here since five years."
Output: "He has been working here for five years."

Every task follows the same pattern: a task-specific text prefix + the input → the desired output as free text.

T5 Architecture Details

Model Configuration by Variant

Variant	Parameters	Layers (Enc/Dec)	Hidden Size	Heads
t5-small	60M	6 / 6	512	8
t5-base	220M	12 / 12	768	12
t5-large	770M	24 / 24	1024	16
t5-3b	3B	24 / 24	1024	32
t5-11b	11B	24 / 24	1024	128

Key Architecture Features

Relative Position Embeddings

Not absolute position encodings — more flexible to different sequence lengths.

Pre-Norm

LayerNorm applied before attention and FFN layers for training stability.

Shared Embeddings

Some variants share input and output embedding matrices to reduce parameters.

T5 Pre-training: Span Corruption

Unlike BERT's single-token masking, T5 masks **contiguous spans** of tokens. The model must generate the missing spans — a more challenging and generation-oriented objective.

```
Original:   "The quick brown fox jumps over the lazy dog"

BERT Mask:  "The quick [MASK] fox jumps [MASK] the lazy dog"
            (single tokens, model predicts one token per [MASK])

T5 Spans:
  Input:  "The quick <X> over the <Y> dog"
  Target: "<X> brown fox jumps <Y> lazy </s>"

Sentinel tokens (<X>, <Y>, ...) replace each corrupted span.
```

Benefits of Span Corruption

Coherent Generation

Forces the model to generate multi-word spans, not just single tokens.

Word Dependencies

Learning to fill spans teaches richer inter-word relationships.

Generation Ready

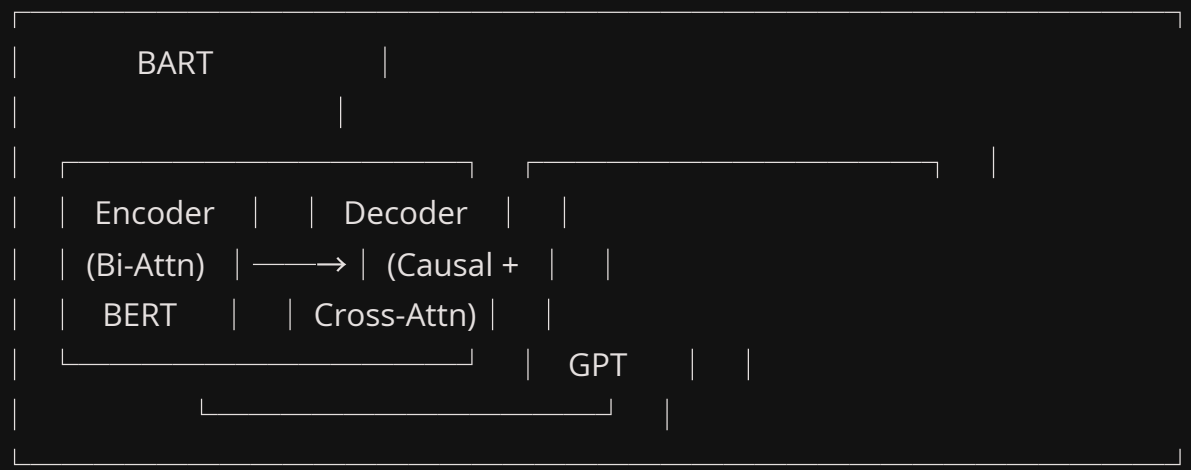
Better pre-training alignment for downstream generation tasks.

More Challenging

Harder than single-token masking → stronger representations.

BART: Bidirectional and Auto-Regressive Transformer

Architecture: BERT Encoder + GPT Decoder



T5 vs BART: When to Use Which?

Feature	T5	BART
Pre-training	Span corruption	Multiple noise types
Task Framing	Task prefix prompts (text-to-text)	Standard seq2seq
Strengths	Multi-task learning, instruction following	Robust reconstruction, text generation
Best for	Multi-task setups, structured prompts	Summarisation, text generation
Model Sizes	Small (60M) to 11B	Base (139M), Large (406M)
Licence	Apache 2.0	Apache 2.0

Quick Decision Guide

Need to handle multiple tasks with task prompts?
→ Use T5

Want best summarization performance?
→ Use BART

Fine-tuning on limited data?
→ BART often better (stronger pre-training objective)

Resource constrained?
→ T5-small (60M) vs BART-base (139M)

Encoder-Decoder Use Cases

1. Translation

```
from transformers import pipeline

# Helsinki-NLP models cover 1000+ language pairs
translator = pipeline("translation",
                      model="Helsinki-NLP/opus-mt-en-fr")
result = translator("Hello, how are you?")
# [{'translation_text': 'Bonjour, comment allez-vous?'}]

# Multilingual: NLLB — No Language Left Behind
translator = pipeline(
    "translation",
    model="facebook/nllb-200-distilled-600M",
    src_lang="eng_Latn",   # FLORES-200 source code
    tgt_lang="fra_Latn"    # FLORES-200 target code
)
```

Language Code Formats

Model	Code Format	Example
Helsinki-NLP	ISO 639-1	en, fr, de, zh
NLLB / M2M-100	FLORES-200	eng_Latn, fra_Latn, zho_Hans

Encoder-Decoder Use Cases (continued)

2. Summarisation with BART

```
from transformers import pipeline

summarizer = pipeline("summarization",
                      model="facebook/bart-large-cnn")

article = """The transformer is a deep learning architecture introduced in
the 2017 paper 'Attention Is All You Need' by Google researchers. Unlike
previous models that processed sequences sequentially, the transformer uses
self-attention mechanisms to process all tokens simultaneously, enabling
much better parallelisation. This breakthrough has led to significant
advances in NLP, including models like BERT, GPT, and T5. The architecture
has since been adapted for computer vision, speech recognition, and
multimodal applications."""

summary = summarizer(article, max_length=50, min_length=20, do_sample=False)
print(summary[0]['summary_text'])
# "The transformer architecture uses self-attention to process sequences
#  in parallel. This has led to advances in NLP models like BERT, GPT, T5."
```

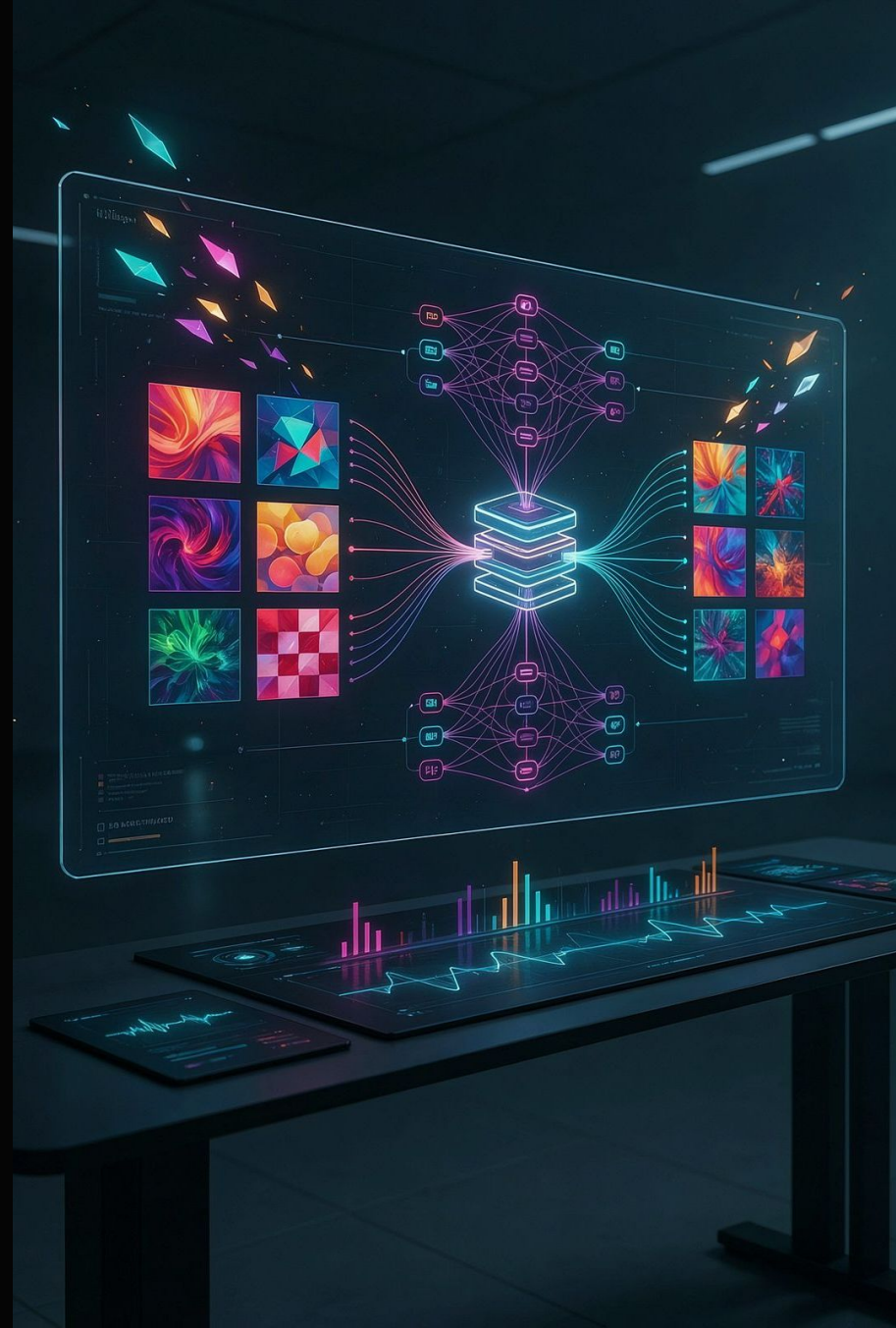
3. Text-to-Text Generation with T5

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained("t5-base")
model = AutoModelForSeq2SeqLM.from_pretrained("t5-base")

# Grammar correction
input_text = "grammar: He have been working here since five years."
inputs = tokenizer(input_text, return_tensors="pt",
                   max_length=512, truncation=True)
outputs = model.generate(**inputs, max_length=128)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
# "He has been working here for five years."

# Question generation
context = "The Eiffel Tower is named after engineer Gustave Eiffel."
input_text = f"generate question: {context}"
# Output: "Who is the Eiffel Tower named after?"
```

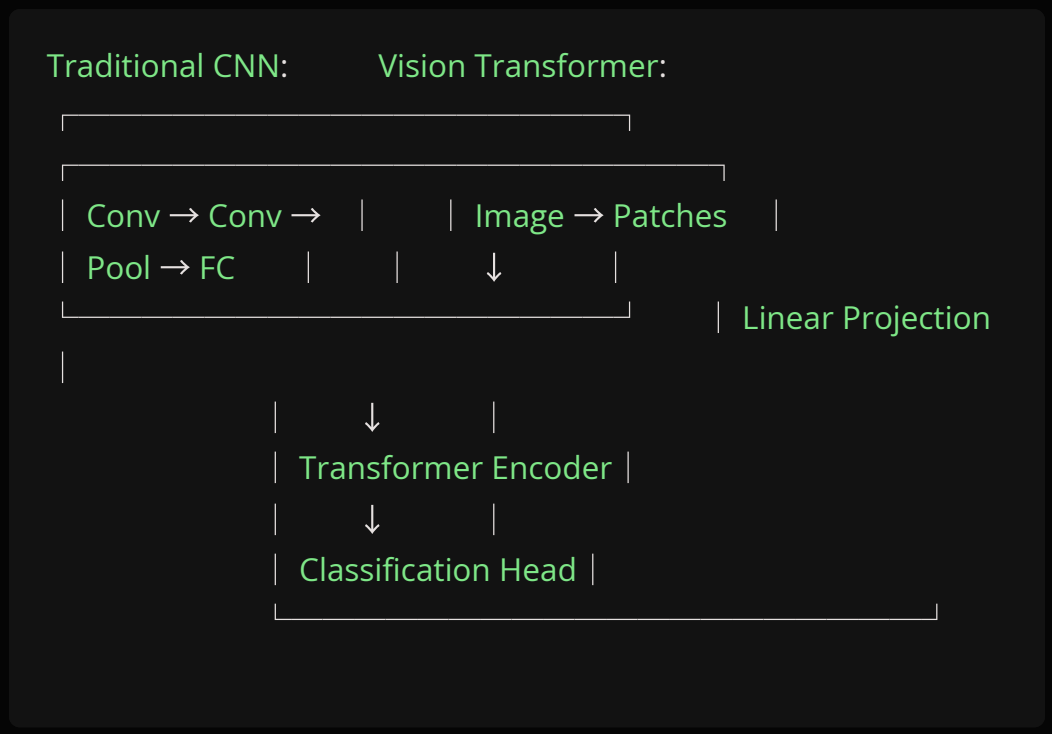
VISION TRANSFORMERS

Vision Transformers (ViT)

ViT applies the same transformer architecture that revolutionised NLP directly to images, by treating **image patches as tokens**. A 224×224 image split into 16×16 patches yields 196 "visual tokens" — fed straight into a standard Transformer Encoder.

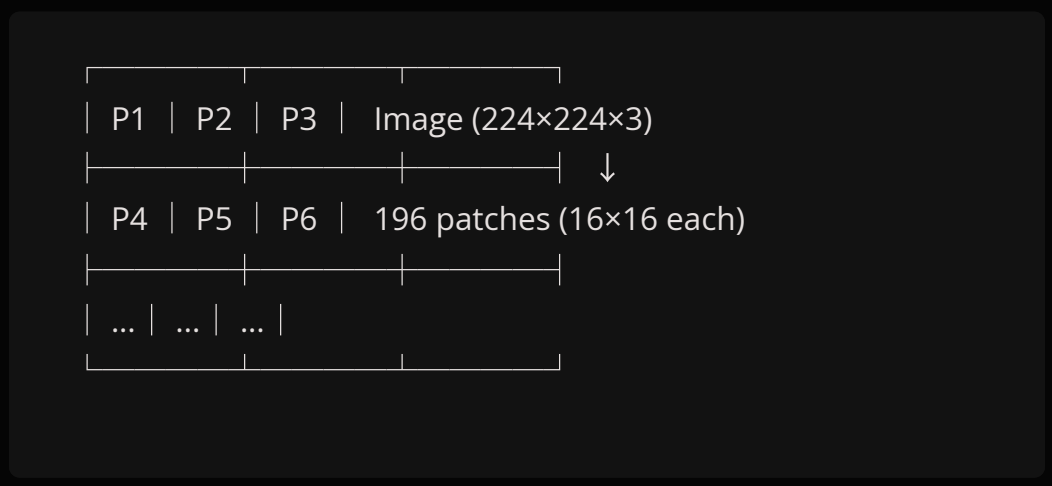


How ViT Works



Step-by-step:

1. Split 224×224 image into 16×16 patches → 196 patches



1. Flatten each patch → linear projection → patch embeddings
2. Prepend a `[CLS]` token + add position embeddings
3. Feed 197-token sequence through Transformer Encoder
4. `[CLS]` output → Classification Head → class prediction

Patch Size Trade-off

Patch 16×16 (ViT-B/16):
196 patches → 196 tokens
More detail, more computation

Patch 32×32 (ViT-B/32):
49 patches → 49 tokens
Less detail, faster

ViT Model Variants

Model	Params	Hidden
ViT-Tiny	5.7M	192
ViT-Small	22M	384
ViT-Base	86M	768
ViT-Large	307M	1024
ViT-Huge	632M	1280

ViT vs CNN: When to Use Which?

Aspect	CNN	Vision Transformer
Inductive Bias	Strong (locality, translation invariance)	Weak — learns structure from data
Data Efficiency	Good on small datasets	Needs more data or pre-training
Performance (large data)	Good	Excellent — often beats CNNs
Computational Cost	Lower	Higher (quadratic in patches)
Interpretability	Feature maps	Attention maps (more intuitive)

Recommendation Guide

→ Small dataset (<100K images)

Use a CNN (ResNet, EfficientNet) or fine-tune a pre-trained ViT on your domain.

→ Large dataset / ImageNet-21K transfer

ViT frequently outperforms CNNs when sufficient data is available for pre-training.

→ Need interpretability

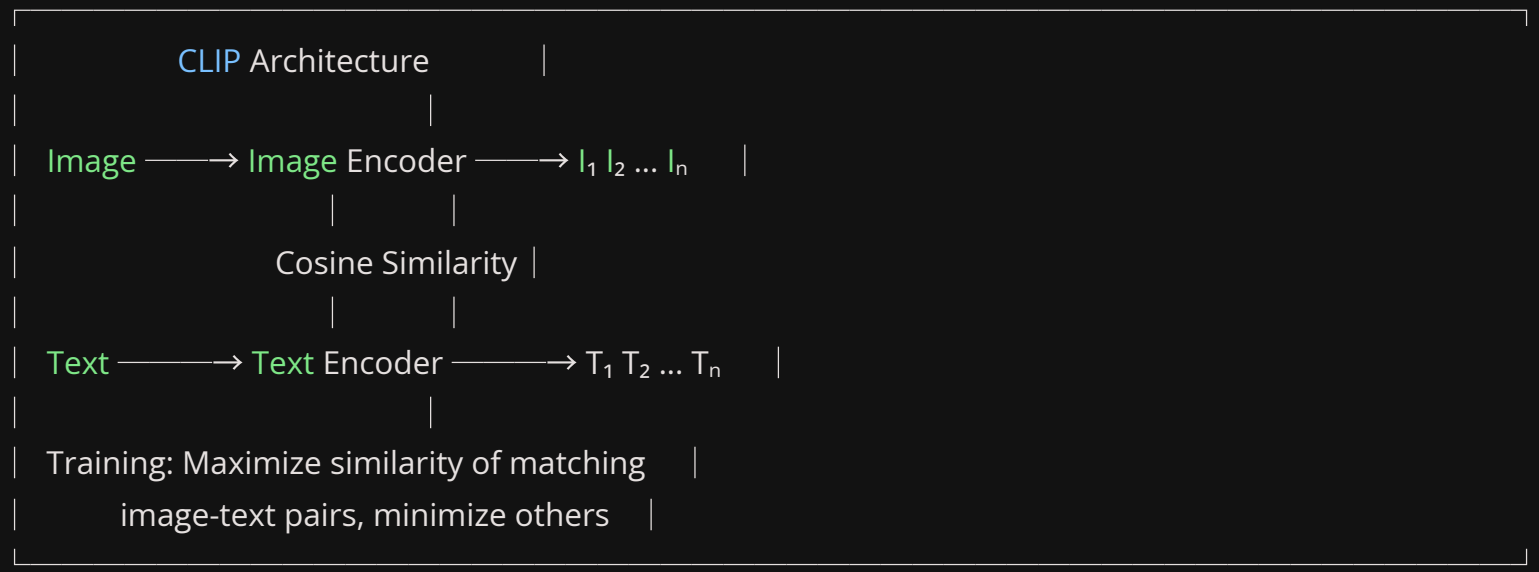
ViT's attention maps are more intuitive than CNN feature maps for understanding model focus.

MULTIMODAL MODELS

CLIP: Connecting Text and Images

CLIP (Contrastive Language-Image Pre-training) trains separate image and text encoders to produce embeddings in a **shared semantic space**. Matching image-text pairs are pulled close; mismatched pairs are pushed apart.

CLIP Architecture & Zero-Shot Classification



Other Multimodal Models

Vision-Language Model Ecosystem

Model	Architecture	Capabilities	Size
CLIP	Dual encoder	Zero-shot classification, retrieval	428M
BLIP-2	Q-Former + LLM	Captioning, Visual QA	3.7B+
LLaVA	ViT + LLM	Visual chat, reasoning	7B+
GIT	Encoder-Decoder	Image captioning	700M
ViLT	Single transformer	VQA (lighter weight)	87M

Image-to-Text (Captioning)

```
from transformers import pipeline

captioner = pipeline("image-to-text",
 model="nlpconnect/vit-gpt2-image-captioning")

# Input: image of a dog playing in the park
# Output: "a dog running in the grass near a park bench"
```

Speech and Audio Models

Whisper: Automatic Speech Recognition

```python
from transformers import pipeline

transcriber = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-tiny"  # 39M params — fastest
)

result = transcriber("audio_file.mp3")
print(result['text'])
```

Whisper Model Variants

Variant	Parameters	Notes
whisper-tiny	39M	Fastest, lowest accuracy
whisper-base	74M	Good balance for quick tasks
whisper-small	244M	Strong multilingual support
whisper-medium	769M	High accuracy
whisper-large	1.5B	Best accuracy, GPU recommended

Other Audio Models

Wav2Vec2

Speech recognition — Meta

Bark

Text-to-speech — Suno

MusicGen

Music generation — Meta

AudioLDM

Text-to-audio generation

HANDS-ON LAB MODULE 4

Lab: Multi-Language Translation

Exercise 1 — Helsinki-NLP Language Pairs

```
from transformers import pipeline
import torch

translator = pipeline(
 "translation",
 model="Helsinki-NLP/opus-mt-en-fr",
 device=0 if torch.cuda.is_available() else -1
)

english_texts = [
 "The weather is beautiful today.",
 "Machine learning is transforming industries.",
 "I would like a cup of coffee, please."
]

for text in english_texts:
 result = translator(text)
 print(f"EN: {text}")
 print(f"FR: {result[0]['translation_text']}\n")

# Explore multiple language pairs
translators = {
 "en→fr": "Helsinki-NLP/opus-mt-en-fr",
 "en→de": "Helsinki-NLP/opus-mt-en-de",
 "en→es": "Helsinki-NLP/opus-mt-en-es",
 "en→it": "Helsinki-NLP/opus-mt-en-it",
 "en→nl": "Helsinki-NLP/opus-mt-en-nl",
}
text = "Hello, how are you?"
for lang_pair, model_name in translators.items():
 t = pipeline("translation", model=model_name)
 result = t(text)
 print(f"{lang_pair}: {result[0]['translation_text']}")
```

Lab: Summarisation with BART

Exercise 2 — Variable Length Summaries

```
from transformers import pipeline

summarizer = pipeline("summarization",
 model="facebook/bart-large-cnn", device=-1)

long_text = """
Artificial intelligence has made remarkable progress in recent years,
particularly in natural language processing. Large language models like
GPT-3, BERT, and their successors have demonstrated impressive capabilities
in understanding and generating human-like text. These models are trained on
massive datasets containing billions of words. Through pre-training, they
learn patterns in language, grammar, and even some reasoning abilities.
However, they can generate plausible-sounding but factually incorrect
information, exhibit biases present in training data, and require enormous
computational resources. Researchers are actively working on making these
models more efficient, accurate, and aligned with human values.
"""

for length_ratio in [0.3, 0.5, 0.7]:
 input_length = len(long_text.split())
 max_len = int(input_length * length_ratio)
 min_len = max_len // 2

 summary = summarizer(long_text,
                      max_length=max_len,
                      min_length=min_len,
                      do_sample=False)

 print(f"\nSummary ({length_ratio:.0%} length, ~{max_len} words):")
 print(summary[0]['summary_text'])
```

Lab: Image Classification with ViT

Exercise 3 — google/vit-base-patch16-224

```
from transformers import pipeline
from PIL import Image
import requests
from io import BytesIO
import matplotlib.pyplot as plt

# Load pipeline
classifier = pipeline("image-classification",
 model="google/vit-base-patch16-224")

# Load sample image
url = ("https://huggingface.co/datasets/huggingface/"
 "documentation-images/resolve/main/pipeline-cat-chonk.jpeg")
response = requests.get(url)
image = Image.open(BytesIO(response.content))

# Display image
plt.imshow(image)
plt.axis('off')
plt.show()

# Classify — returns top-5 predictions with scores
results = classifier(image)
for result in results[:5]:
 print(f"{result['label']}: {result['score']:.3f}")

# Swap in your own image:
# image = Image.open("path/to/your/image.jpg")
# results = classifier(image)
```

Lab: Zero-Shot Classification with CLIP

Exercise 4 — Standard and Creative Labels

```
from transformers import pipeline
import requests
from PIL import Image
from io import BytesIO

# Load pipeline
classifier = pipeline("zero-shot-image-classification",
 model="openai/clip-vit-base-patch32")

url = ("https://huggingface.co/datasets/huggingface/"
 "documentation-images/resolve/main/pipeline-cat-chonk.jpeg")
response = requests.get(url)
image = Image.open(BytesIO(response.content))

# Standard labels
candidate_labels = [
 "a cat", "a dog", "a tiger", "a lion",
 "a rabbit", "a car", "a building", "food"
]
results = classifier(image, candidate_labels=candidate_labels)
for result in results:
 print(f"{result['label']}: {result['score']:.3f}")

# Creative / abstract labels — CLIP is not limited to training classes!
creative_labels = [
 "a majestic feline", "an internet meme star",
 "a sleepy animal", "a predator", "someone's pet"
]
results = classifier(image, candidate_labels=creative_labels)
print("\nCreative labelling:")
for result in results:
 print(f"{result['label']}: {result['score']:.3f}")
```

Lab: Speech Recognition with Whisper

Exercise 5 — Setup and LibriSpeech Sample

```
from transformers import pipeline
from datasets import load_dataset
import torch

# Load ASR pipeline
transcriber = pipeline(
 "automatic-speech-recognition",
 model="openai/whisper-tiny",
 device=0 if torch.cuda.is_available() else -1
)

# Using a local audio file:
# result = transcriber("audio.mp3")
# print(result["text"])

print("Whisper models available:")
print(" whisper-tiny: 39M parameters (fastest)")
print(" whisper-base: 74M parameters")
print(" whisper-small: 244M parameters")
print(" whisper-medium: 769M parameters")
print(" whisper-large: 1.5B parameters (best quality)")

# Demo with LibriSpeech sample
print("\nLoading sample from LibriSpeech...")
try:
 dataset = load_dataset("librispeech_asr", "clean",
 split="test", streaming=True)
 sample = next(iter(dataset))
 result = transcriber(sample["audio"]["array"])
 print(f"Transcription: {result['text']}")
 print(f"Reference: {sample['text']}")
except Exception as e:
 print(f"Couldn't load sample: {e}")
 print("Install librosa and soundfile to enable audio loading.")
```

Lab: Multi-Model Pipeline Chain

Exercise 6 — Image → Caption → Translation → Sentiment

```
from transformers import pipeline
from PIL import Image
import requests
from io import BytesIO

# Build the three-stage chain
captioner = pipeline("image-to-text",
 model="nlpconnect/vit-gpt2-image-captioning")

translator = pipeline("translation_en_to_fr",
 model="Helsinki-NLP/opus-mt-en-fr")

sentiment_analyzer = pipeline("sentiment-analysis",
 model="distilbert-base-uncased-finetuned-sst-2-english")

# Load image
url = ("https://huggingface.co/datasets/huggingface/"
 "documentation-images/resolve/main/pipeline-cat-chonk.jpeg")
response = requests.get(url)
image = Image.open(BytesIO(response.content))

# Stage 1: Generate caption
print("\n1. Generating caption...")
caption = captioner(image)[0]['generated_text']
print(f" Caption: {caption}")

# Stage 2: Translate to French
print("\n2. Translating to French...")
translated = translator(caption)[0]['translation_text']
print(f" French: {translated}")

# Stage 3: Sentiment of the caption
print("\n3. Analysing sentiment...")
sentiment = sentiment_analyzer(caption)[0]
print(f" Sentiment: {sentiment['label']} "
 f"(confidence: {sentiment['score']:.3f})")

print("\n" + "="*50)
print(f"Input: Image of a cat")
print(f"→ Caption: {caption}")
print(f"→ French: {translated}")
print(f"→ Sentiment: {sentiment['label']}")
```

MODULE 4 SUMMARY

Key Takeaways — Module 4

Encoder-Decoder models

T5 and BART combine bidirectional understanding with autoregressive generation. Perfect for tasks where input and output lengths differ: translation, summarisation, and text transformation.

Vision Transformers (ViT)

Images become token sequences via patch splitting. ViT is competitive with or better than CNNs on large datasets and offers more interpretable attention maps.

Multimodal models

CLIP enables zero-shot image classification by comparing image and text embeddings. Image captioning, visual QA, and speech recognition (Whisper) extend transformers far beyond NLP.

Pipeline chaining

HuggingFace pipelines can be chained to build complex multi-model workflows — e.g. image → caption → translation → sentiment — with minimal code.

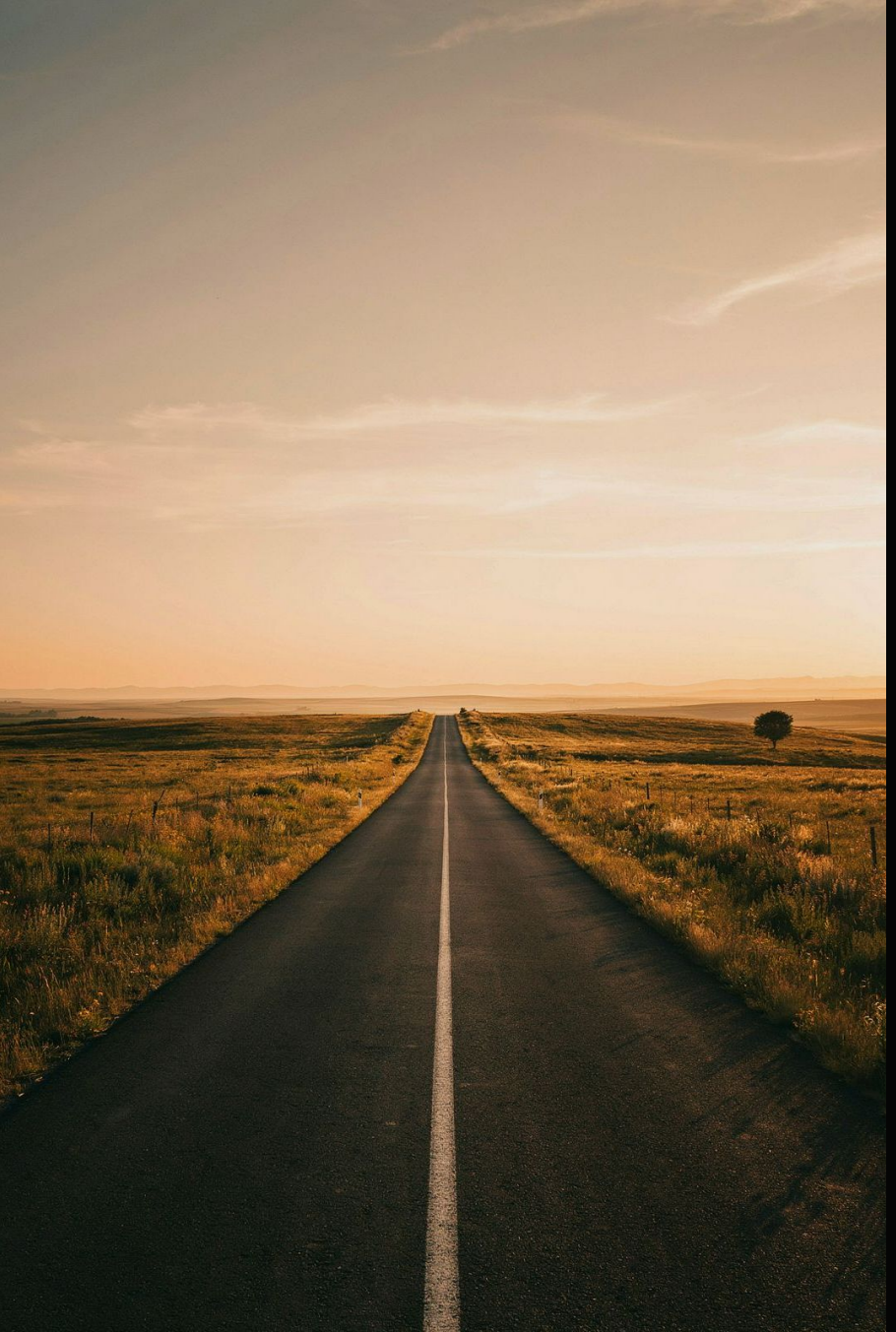
Day 2 Complete Recap

Module 3 — Model Architectures

- Encoder-only models: BERT, RoBERTa, DistilBERT, ALBERT, DeBERTa, ModernBERT
- Decoder-only models: GPT-2, SmolLM2, Phi, Gemma
- Feature extraction and contextual embeddings
- Text generation strategies: greedy, beam search, sampling with temperature/top-k/top-p

Module 4 — Multimodal & Seq2Seq

- Encoder-Decoder models: T5 and BART
- Translation, summarisation, and text-to-text tasks
- Vision Transformers (ViT) — images as patch sequences
- Multimodal models: CLIP, image captioning
- Speech recognition with Whisper



COMING UP

Tomorrow — Day 3 Preview

The Trainer API

HuggingFace's high-level fine-tuning API — configure training arguments, automatic evaluation, and checkpoint saving with minimal boilerplate.

Data Preparation & Evaluation

Tokenising datasets, building DataCollators, and using `evaluate` metrics for task-specific model assessment.

Custom Training Loops

When the Trainer API isn't enough — writing your own PyTorch training loop with gradient accumulation and mixed precision.

Full Fine-tuning on Custom Datasets

End-to-end walkthrough: load data, tokenise, fine-tune a pre-trained model, evaluate, and save for deployment.

ADDITIONAL RESOURCES | MODULE 3

Resources — Module 3

📄 Key Papers

BERT: Pre-training of Deep Bidirectional Transformers

Devlin et al., 2018 — the foundational BERT paper.

RoBERTa: A Robustly Optimised BERT Pretraining Approach

Liu et al., 2019 — improved training recipe for BERT.

DistilBERT: A Distilled Version of BERT

Sanh et al., 2019 — knowledge distillation for efficiency.

ModernBERT Blog Post

HuggingFace, 2024 — introducing BERT's modern successor.

🎓 Tutorials & Tools

HuggingFace Course — Chapter 2: Using Transformers

Official hands-on tutorial covering pipelines and the model API.

How to Generate Text

Deep dive into generation strategies, decoding algorithms, and parameters.

BERTViz: Visualise Attention

Interactive tool to explore BERT's attention patterns across layers and heads.

Tokeniser Playground

Live demo showing how different tokenisers split text into sub-word tokens.

ADDITIONAL RESOURCES | MODULE 4

Resources — Module 4

📄 Key Papers

T5: Exploring the Limits of Transfer Learning

Raffel et al., 2019 — the text-to-text transfer transformer.

BART: Denoising Sequence-to-Sequence Pre-training

Lewis et al., 2019 — denoising autoencoder for generation tasks.

An Image is Worth 16×16 Words: Vision Transformer

Dosovitskiy et al., 2020 — the original ViT paper.

CLIP: Learning Transferable Visual Models

Radford et al., 2021 — contrastive image-language pre-training.

🎓 Tutorials & Demos

HuggingFace Course — Chapter 3: Fine-tuning

Official tutorial for fine-tuning pre-trained models on custom data.

Fine-tuning ViT Tutorial

Step-by-step guide for adapting ViT to custom image classification tasks.

Whisper Fine-tuning Guide

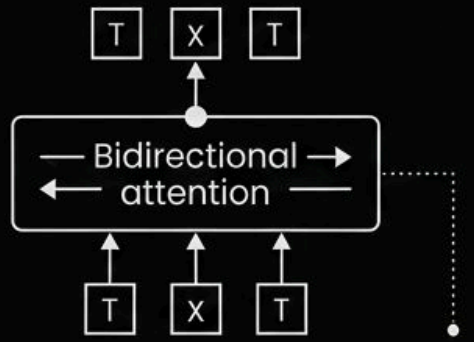
End-to-end guide for fine-tuning Whisper on custom speech data.

CLIP Zero-Shot Demo

Interactive demo — classify any image with your own text labels.

Architecture Summary at a Glance

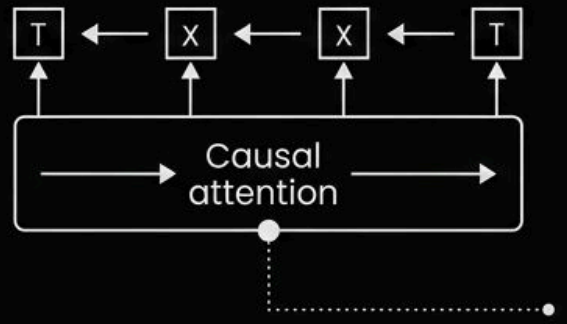
Encoder-Only (BERT family)



Ideal for classification, NER, embeddings.

Key models: BERT RoBERTa DistilBERT ModernBERT

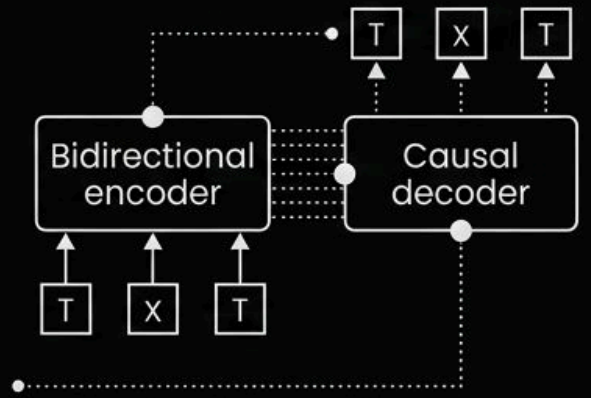
Decoder-Only (GPT family)



Ideal for text generation, completion, chat.

Key models: GPT-2 SmolLM2 Phi Gemma

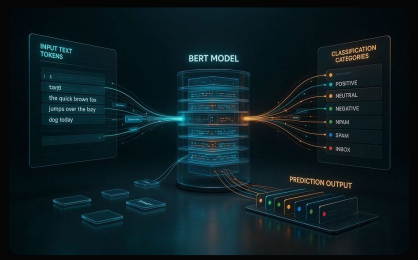
Encoder-Decoder (T5/BART family)



Ideal for translation, summarisation, seq2seq.

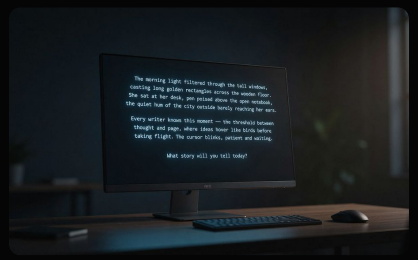
Key models: T5 BART

The Transformer Ecosystem Beyond NLP



Text Understanding

Encoder-only models (BERT family) dominate tasks requiring deep comprehension: classification, NER, question answering, and dense retrieval embeddings.

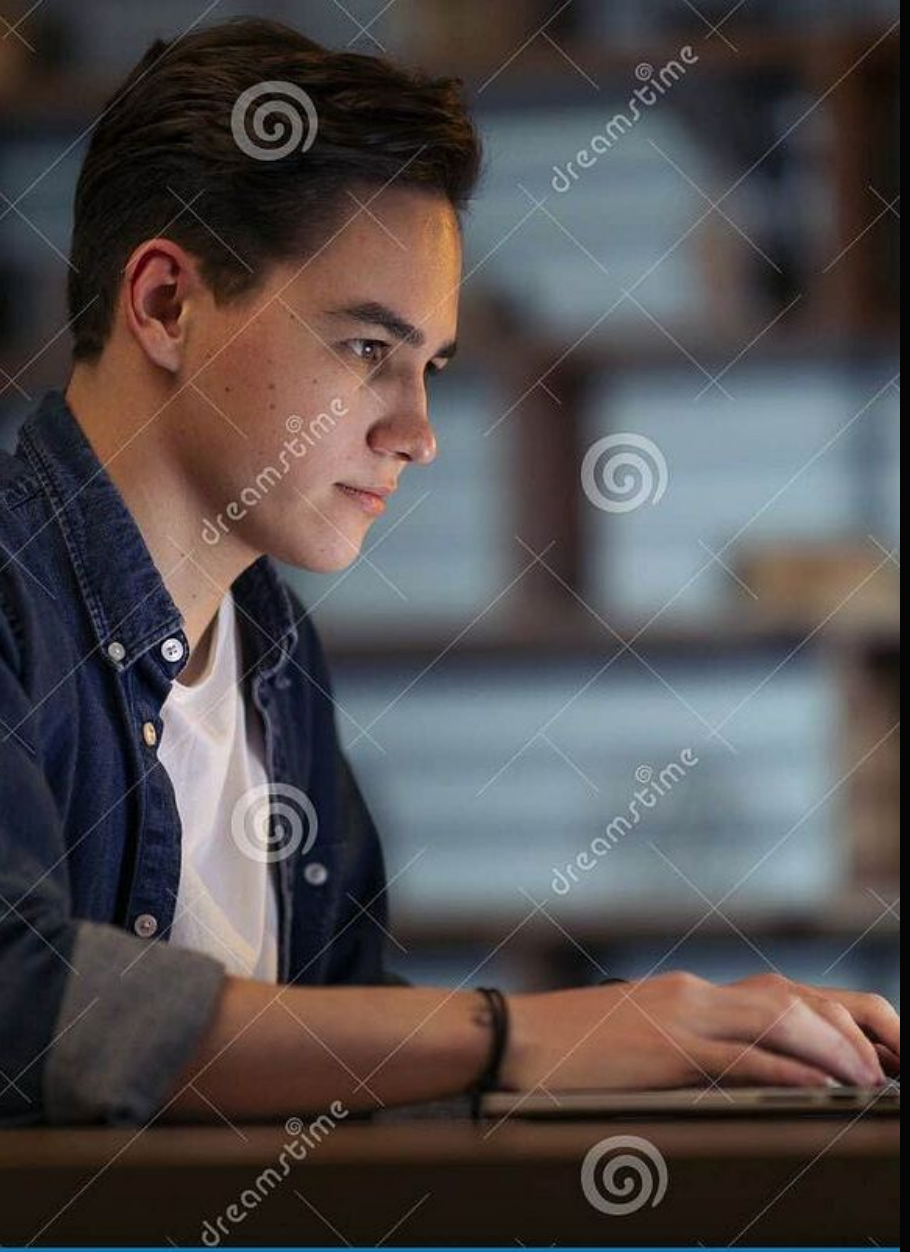


Text Generation

Decoder-only models (GPT family) power chat assistants, code completion, creative writing, and instruction following at every scale from 100M to 100B+ parameters.

Vision & Multimodal

ViT, CLIP, and vision-language models have extended the transformer paradigm to images, video, audio, and cross-modal understanding tasks.



That's a Wrap — Day 2

You now have a thorough grounding in the **full spectrum of Transformer model architectures** — from BERT's bidirectional encoders to GPT's causal decoders, T5 and BART's sequence-to-sequence frameworks, and the multimodal frontier with ViT and CLIP.

🔧 Day 3: Fine-tuning

The Trainer API · Data pipelines · Custom training loops · Deploying fine-tuned models

💡 Practice Tonight

Run the lab exercises · Try your own prompts · Experiment with different generation parameters · Chain a pipeline

Checkout the updated GitHub repo:
https://github.com/chandrashekar-babu/huggingface-transformers